

HLD Case Study 9: Real-Time Ride Dispatch Platform (Uber / Lyft)

Key Focus Areas: Real-time location tracking at massive ingestion volume (1M+ writes/sec), Uber H3 hexagonal spatial indexing, dispatch matching state machines, and dynamic surge pricing.

1. Problem Statement & Functional Requirements

Design the real-time location streaming and driver dispatch architecture for a global ride-hailing platform.

Requirements:

- High-Frequency Location Tracking:** 5 Million active drivers streaming GPS coordinates every 4 seconds.
- Nearest Driver Dispatch:** Match riders with optimal nearby drivers within 5 seconds.
- Trip State Machine:** Manage the trip lifecycle from Requested $\rightarrow$ Dispatched $\rightarrow$ Arrived $\rightarrow$ In-Trip $\rightarrow$ Completed.
- Dynamic Surge Pricing:** Adjust fares based on regional supply/demand imbalances.

2. Ingestion Scale & Estimations

- Active Drivers:** 5 Million concurrent vehicles worldwide.
- Ping Frequency:** 1 ping every 4 seconds.
- Location Ingestion QPS:** $\frac{5,000,000}{4} = 1,250,000 \text{ Writes/Second!}$
- Storage Consideration:** A relational database would crash immediately under 1.25M QPS of writes. Location pings are **ephemeral**; we only need the latest driver position in RAM!

3. High-Level Architecture Diagram

flowchart TD

```
Drivers["5 Million Active Drivers"] -->|GPS pings every 4s (gRPC / WebSocket)| NetLB[NetLB]
NetLB --> LocationIngest["Location Ingestion Gateway Fleet (Netty / Go)"]
```

```
LocationIngest --> KafkaStream["Kafka: driver_location_stream<br/>(Partitioned by H3)"]
```

```
KafkaStream --> SpatialIndexWorker["Spatial Indexing Worker Fleet"]
```

```
SpatialIndexWorker --> RedisSpatial["In-Memory Spatial Store (Redis Cluster / H3)<br/>"]
```

```
Rider["Rider App"] --> DispatchSvc["Dispatch & Matching Service"]
```

```
DispatchSvc --> RedisSpatial
```

```
DispatchSvc --> MatchWorker["Dispatch Offer Worker (15s Acceptance Timer)"]
```

```
MatchWorker --> TripDB["Trips Database (PostgreSQL / CockroachDB)"]
```

4. The Dispatch & Matching Sequence

```
sequenceDiagram
    autonumber
    actor Rider as Rider App
    participant Dispatch as Dispatch Service
    participant Spatial as Redis H3 Spatial Index
    participant Kafka as Kafka / Timer Engine
    actor Driver as Driver App

    Rider->>Dispatch: request_ride(pickup_lat, pickup_lon)
    Note over Dispatch: 1. Convert pickup (lat, lon) to H3 Hex Cell (Resolution 8)
    Dispatch->>Spatial: Find active drivers in H3 Cell + adjacent ring of 6 hexagons
    Spatial->>Dispatch: Returns candidate drivers: [D1, D2, D3]
    Note over Dispatch: 2. Rank candidates by ETA and rating; select best driver D1
    Dispatch->>Kafka: create_dispatch_offer(trip_id, driver_id=D1, timeout=15s)
    Kafka->>Driver: Push: "New Ride Request (15 seconds to accept)"

    alt Driver Accepts within 15s
        Driver->>Dispatch: accept_offer(trip_id)
        Dispatch->>Spatial: Mark Driver D1 as BUSY
        Dispatch->>Rider: Driver Found! Arriving in 4 mins.
    else Driver Rejects or 15s Timer Expires
        Note over Dispatch: Re-route offer to next candidate D2!
    end
end
```